

Registration Form

Enter Name

Enter Email

Enter Password

Register

Registration Successful!

Registered Users

- ABCD - abcd@gmail.com

Outcome

API data is successfully fetched and displayed, and the registration form performs proper validation.

Experiment 10

Title:

Java OOP Concepts Implementation

Problem Statement:

Create Java classes demonstrating **encapsulation, inheritance, and abstraction**.

Create a **Bank Management System** application where:

1. Create an **abstract class BankAccount** with:
 - accountNumber
 - accountHolderName
 - balance
 - Abstract method: calculateInterest()
2. Create two subclasses:
 - SavingsAccount
 - CurrentAccount
3. Implement:
 - **Encapsulation** (private data members with getters and setters)
 - **Inheritance** (SavingsAccount and CurrentAccount extend BankAccount)
 - **Abstraction** (abstract method implemented in subclasses)
4. Create a **main class** to:
 - Create objects
 - Deposit money

- Display account details
- Calculate interest

Objectives:

After completing this experiment, students will be able to:

- Understand **Object-Oriented Programming principles**
- Implement **Encapsulation using private data members**
- Implement **Inheritance using extends keyword**
- Implement **Abstraction using abstract classes**
- Create **real-world object relationships**

Theory:

Object-Oriented Programming (OOP) is a programming paradigm based on objects and classes. It helps in creating modular, reusable, and maintainable software systems.

Encapsulation

Encapsulation means **wrapping data and methods together into a single unit (class)** and restricting direct access to data using **private variables and getter/setter methods**.

Inheritance

Inheritance allows one class to **acquire properties and behaviors of another class** using the **extends keyword**.

Abstraction

Abstraction hides implementation details and shows only essential features using **abstract classes and methods**.

In this experiment, a **BankAccount abstract class** is created and extended by **SavingsAccount and CurrentAccount** classes.

Algorithm:

1. Start the program.
2. Create an abstract class **BankAccount** with variables:
 - accountNumber
 - accountHolderName
 - balance
3. Declare an abstract method **calculateInterest()**.
4. Create a **SavingsAccount class** extending BankAccount.
5. Implement the **calculateInterest()** method.
6. Create a **CurrentAccount class** extending BankAccount.
7. Implement the **calculateInterest()** method.
8. Create a **Main class**.
9. Create objects for SavingsAccount and CurrentAccount.
10. Perform operations:
 - Deposit money
 - Display account details

- Calculate interest

11. Display results on the console.

12. End the program.

Program Code:

Exp10.java

```
// Abstract class
abstract class BankAccount {
    int accountNumber;
    String accountHolderName;
    double balance;
    // Constructor
    BankAccount(int accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }
    // Deposit Method
    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited: " + amount);
    }
    // Display Account Details
    void displayDetails() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder Name: " + accountHolderName);
        System.out.println("Balance: " + balance);
    }
    // Abstract Method
    abstract void calculateInterest();
}
// SavingsAccount class
class SavingsAccount extends BankAccount {

    SavingsAccount(int accountNumber, String accountHolderName, double balance) {
        super(accountNumber, accountHolderName, balance);
    }
    void calculateInterest() {
        double interest = balance * 0.05; // 5% interest
        System.out.println("Savings Account Interest: " + interest);
    }
}
// CurrentAccount class
class CurrentAccount extends BankAccount {

    CurrentAccount(int accountNumber, String accountHolderName, double balance) {
        super(accountNumber, accountHolderName, balance);
    }
}
```

```

    }
    void calculateInterest() {
        double interest = balance * 0.02; // 2% interest
        System.out.println("Current Account Interest: " + interest);
    }
}
// Main class
public class Exp10 {
    public static void main(String[] args) {
        // Creating objects
        SavingsAccount sa = new SavingsAccount(101, "Rahul", 10000);
        CurrentAccount ca = new CurrentAccount(102, "Anita", 20000);
        System.out.println("----- Savings Account -----");
        sa.deposit(2000);
        sa.displayDetails();
        sa.calculateInterest();
        System.out.println();
        System.out.println("----- Current Account -----");
        ca.deposit(3000);
        ca.displayDetails();
        ca.calculateInterest();
    }
}

```

Expected Outcome:

```

● PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp> javac Exp10.java
● PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp> java Exp10
----- Savings Account -----
Amount Deposited: 2000.0
Account Number: 101
Account Holder Name: Rahul
Balance: 12000.0
Savings Account Interest: 600.0

----- Current Account -----
● Amount Deposited: 3000.0
Account Number: 102
Account Holder Name: Anita
Balance: 23000.0
Current Account Interest: 460.0
○ PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp>

```

Outcome:

Students successfully implemented **encapsulation, inheritance, and abstraction** concepts in Java by developing a simple **Bank Management System** application.

Experiment 11

Title:

Creating REST API using Spring Boot

Problem Statement:

Create a **Spring Boot application** and expose **REST endpoints** using **@RestController**.
Create a **Student Management REST API** that:

1. Returns a **welcome message**.
2. Returns **student details using GET request**.
3. Accepts **student data using POST request**.

Objectives:

After completing this experiment, students will be able to:

- Create a **Spring Boot project**
- Understand **REST architecture**
- Use **@RestController** annotation
- Create **GET and POST endpoints**
- Test APIs using **browser or Postman**

Theory:

Spring Boot is a Java framework used to build **standalone and production-ready applications** quickly. A **REST API (Representational State Transfer API)** allows communication between client and server using HTTP methods.

Common HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Send new data
PUT	Update existing data
DELETE	Remove data

@RestController

@RestController is used to create RESTful web services. It combines:

- @Controller
- @ResponseBody

This allows methods to return **JSON responses directly**. Spring Boot also provides **auto-configuration**, which reduces manual configuration.

Algorithm:

1. Start the Spring Boot project using **Spring Initializer**.
2. Add required dependencies:

- Spring Web
- 3. Create a main application class.
- 4. Create a **StudentController class**.
- 5. Annotate the class with `@RestController`.
- 6. Create a **GET endpoint** to return a welcome message.
- 7. Create another **GET endpoint** to return student details.
- 8. Create a **POST endpoint** to accept student data.
- 9. Run the Spring Boot application.
- 10. Test endpoints using:
 - Browser
 - Postman
- 11. Verify JSON responses.
- 12. Stop the application.

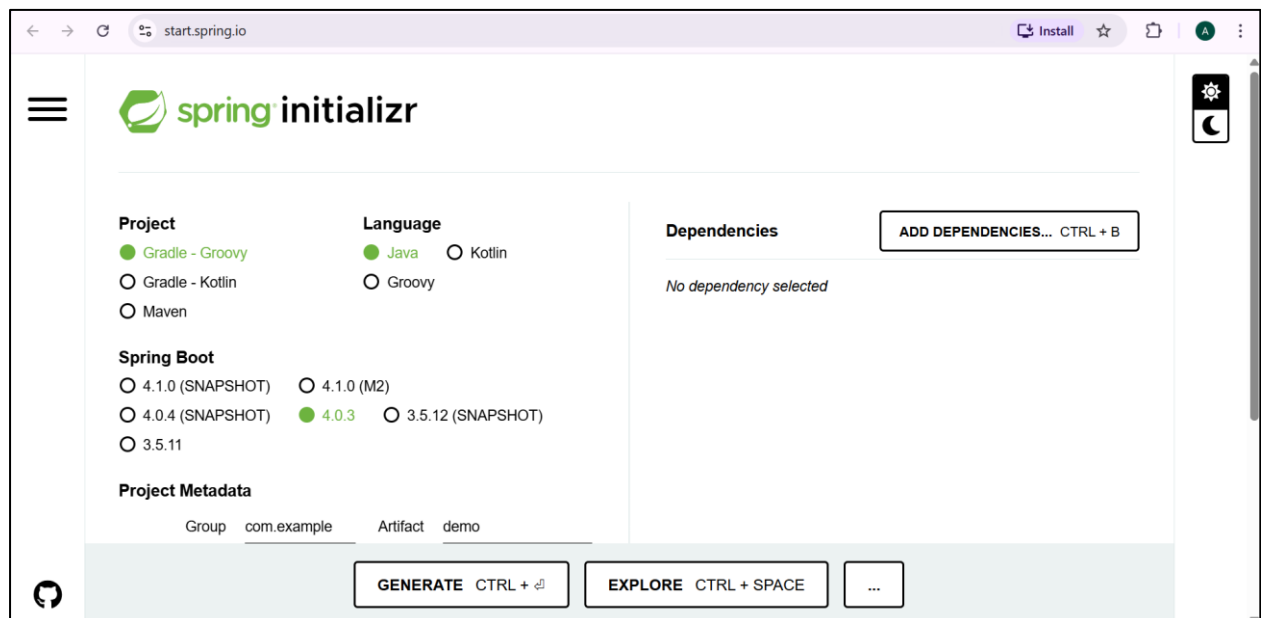
Program Code:

Steps to Create Project using Spring Initializr

Step 1 Open Spring Initializr

Go to:

<https://start.spring.io>



Spring Initializr webpage

Step 2 Configure Project

Project Settings:

Field	Value
Project	Maven

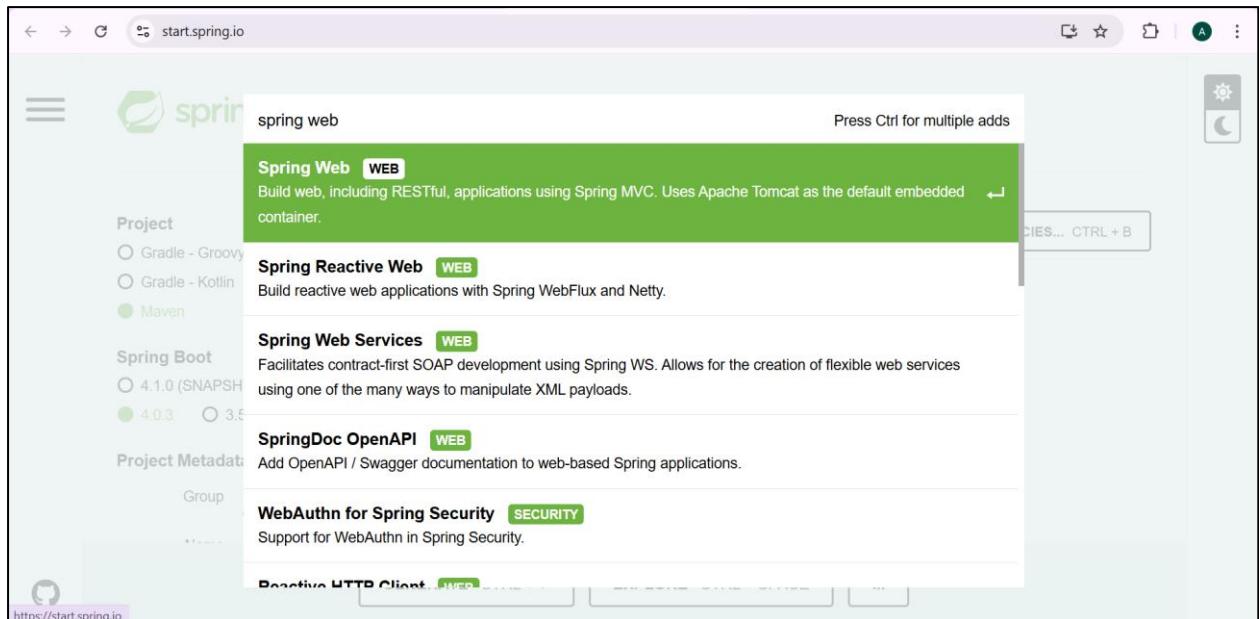
Language Java
Spring Boot Latest stable
Group com.example
Artifact Student-rest-api
Packaging Jar
Java 17

The screenshot shows the Spring Initializr web application at start.spring.io. The interface is divided into several sections:
- **Project:** Radio buttons for Gradle - Groovy, Gradle - Kotlin, and Maven (selected).
- **Language:** Radio buttons for Java (selected), Kotlin, and Groovy.
- **Spring Boot:** Radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (M2), 4.0.4 (SNAPSHOT), 4.0.3 (selected), 3.5.12 (SNAPSHOT), and 3.5.11.
- **Project Metadata:** Text input fields for Group (com.example), Artifact (demo), and Name (demo).
- **Dependencies:** A section with the text "No dependency selected" and a button "ADD DEPENDENCIES... CTRL + B".
At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and an ellipsis button.

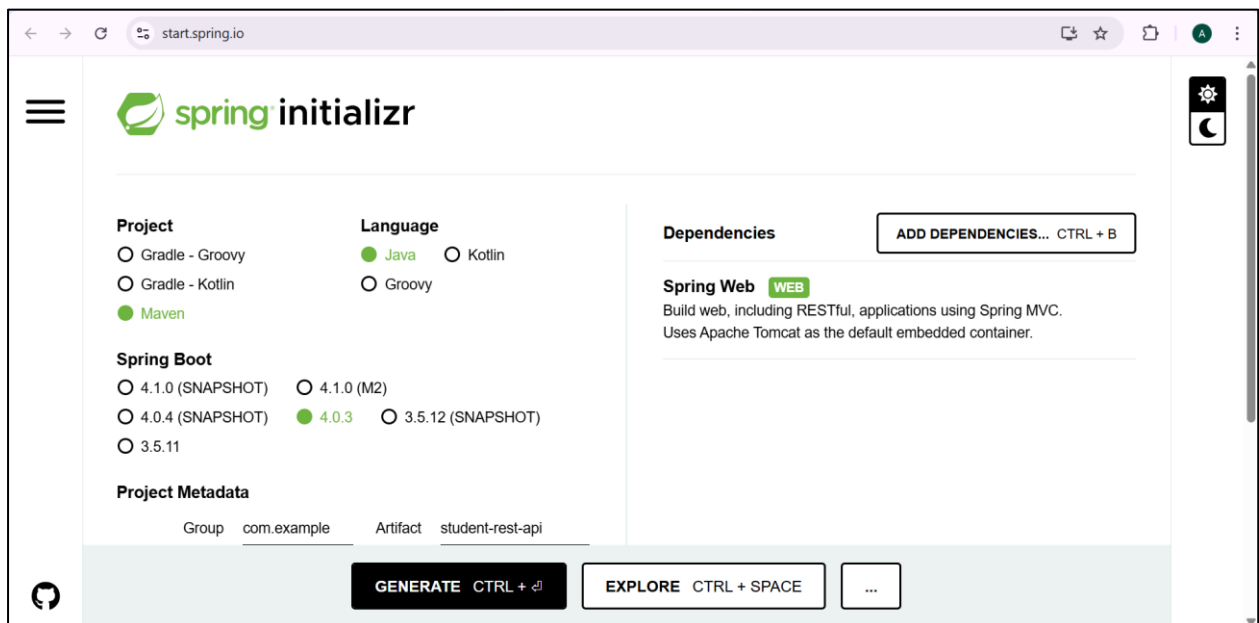
The screenshot shows the second step of the Spring Initializr configuration. The **Spring Boot** section remains the same with 4.0.3 selected. The **Project Metadata** section is expanded to include:
- **Group:** com.example
- **Artifact:** student-rest-api
- **Name:** student-rest-api
- **Description:** Project for Spring Boot
- **Package name:** com.example.studen (partially visible)
- **Packaging:** Radio buttons for Jar (selected) and War.
- **Configuration:** Radio buttons for Properties (selected) and YAML.
The bottom buttons "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and the ellipsis button are still present.

Add Dependency:

Spring Web



Click **Generate** to download project.



Project configuration page

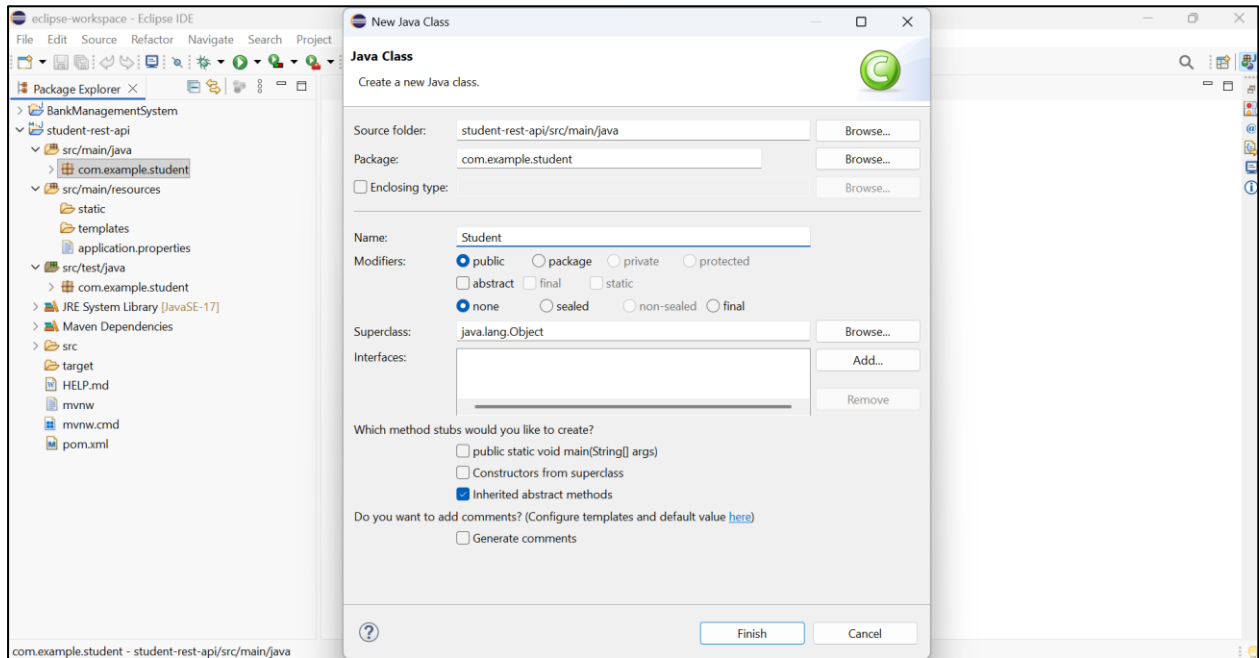
Step 3 Import Project in Eclipse IDE

1. Open Eclipse
2. Click

File → Import

Program Code

Create Student class



Write code inside it:

```
package com.example.student;

// Model class representing Student entity
public class Student {

    private int id;
    private String name;
    private String course;

    // Default constructor
    public Student() {
    }

    // Parameterized constructor
    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    // Getter and Setter methods
    public int getId() {
        return id;
    }
}
```

```

    }

    public void setId(int id) {
        this.id = id;
    }

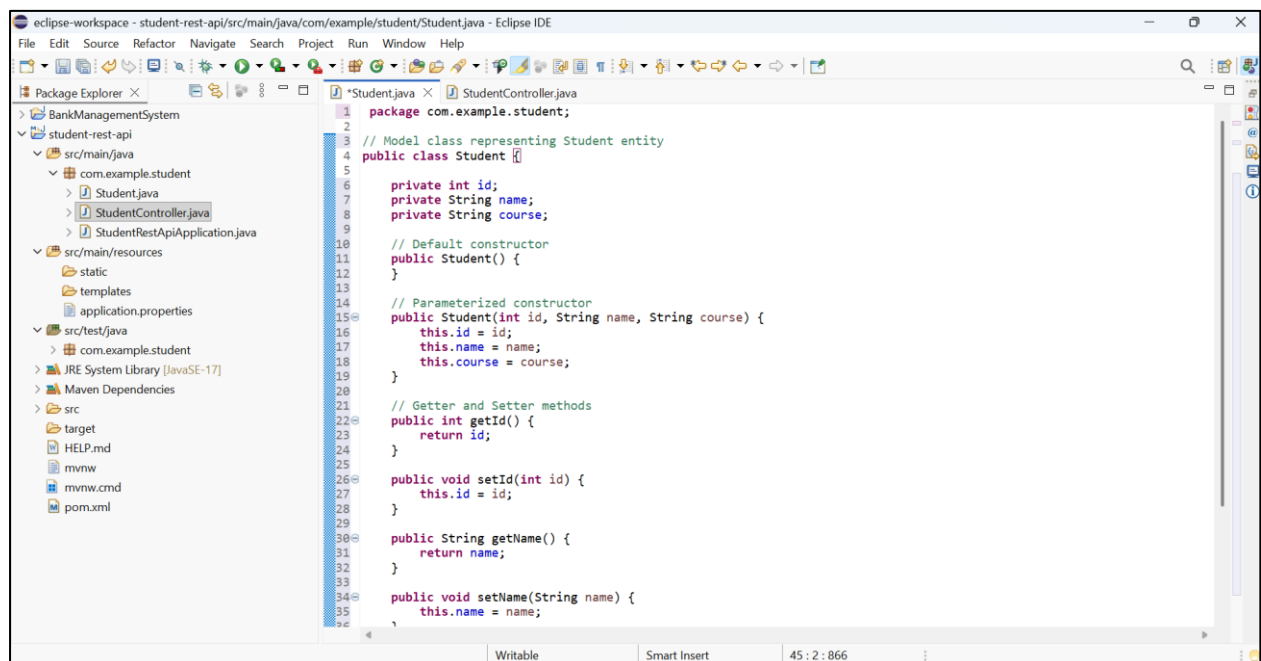
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

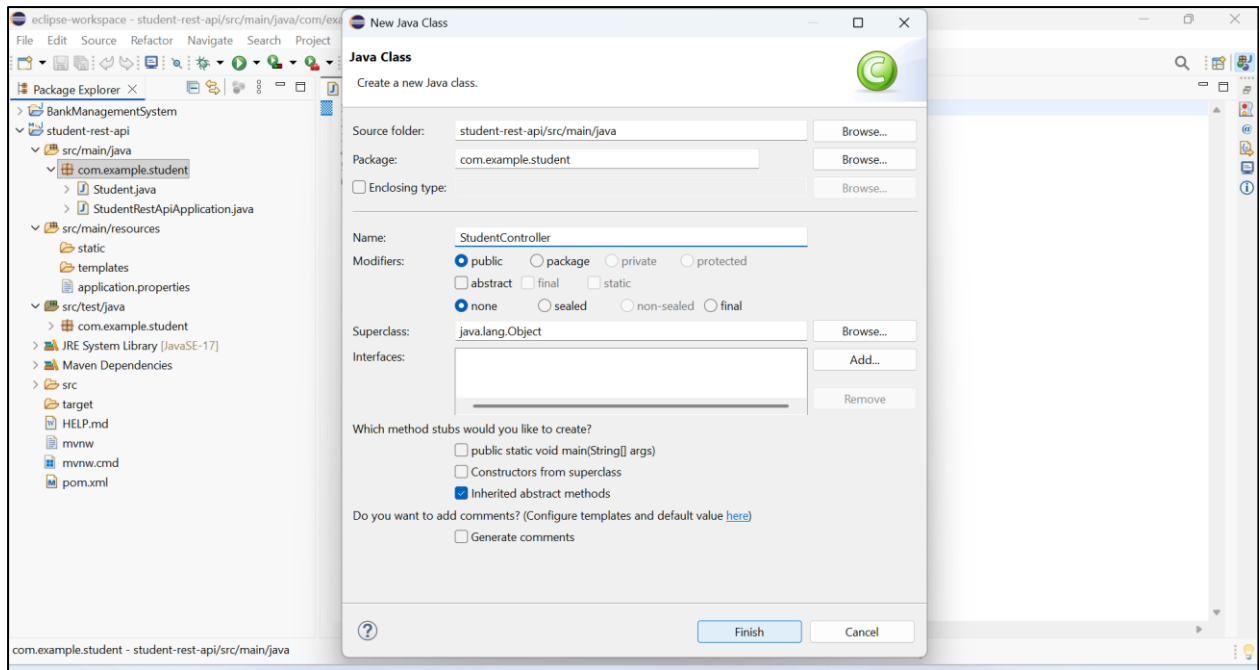
    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}

```



Now Create next class StudentController



Write code inside it:

```
package com.example.student;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// REST Controller
@RestController
@RequestMapping("/")
public class StudentController {

    private Student student;

    // Welcome endpoint
    @GetMapping
    public String welcomeMessage() {
        return "Welcome to Student Management REST API";
    }

    // GET student details
    @GetMapping("/student")
    public ResponseEntity<Student> getStudent() {

        if (student == null) {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }
}
```

```

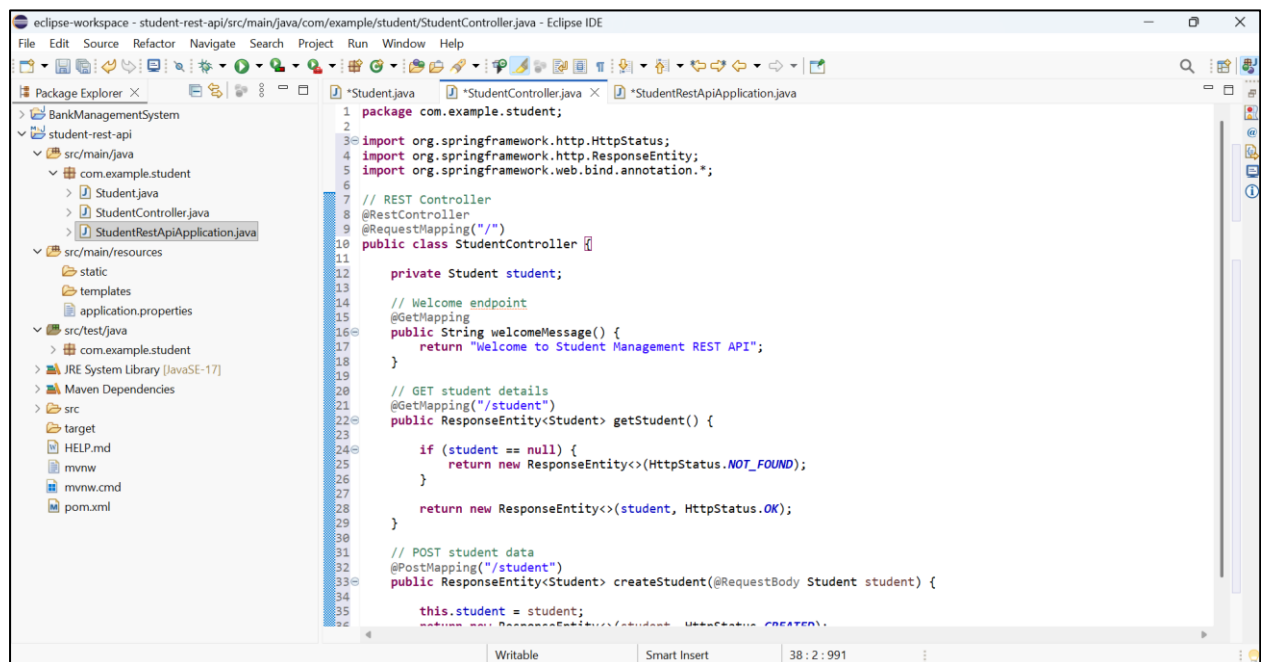
    }

    return new ResponseEntity<>(student, HttpStatus.OK);
}

// POST student data
@PostMapping("/student")
public ResponseEntity<Student> createStudent(@RequestBody Student student) {

    this.student = student;
    return new ResponseEntity<>(student, HttpStatus.CREATED);
}
}

```



Now create Main Application

Create class StudentRestApiApplication

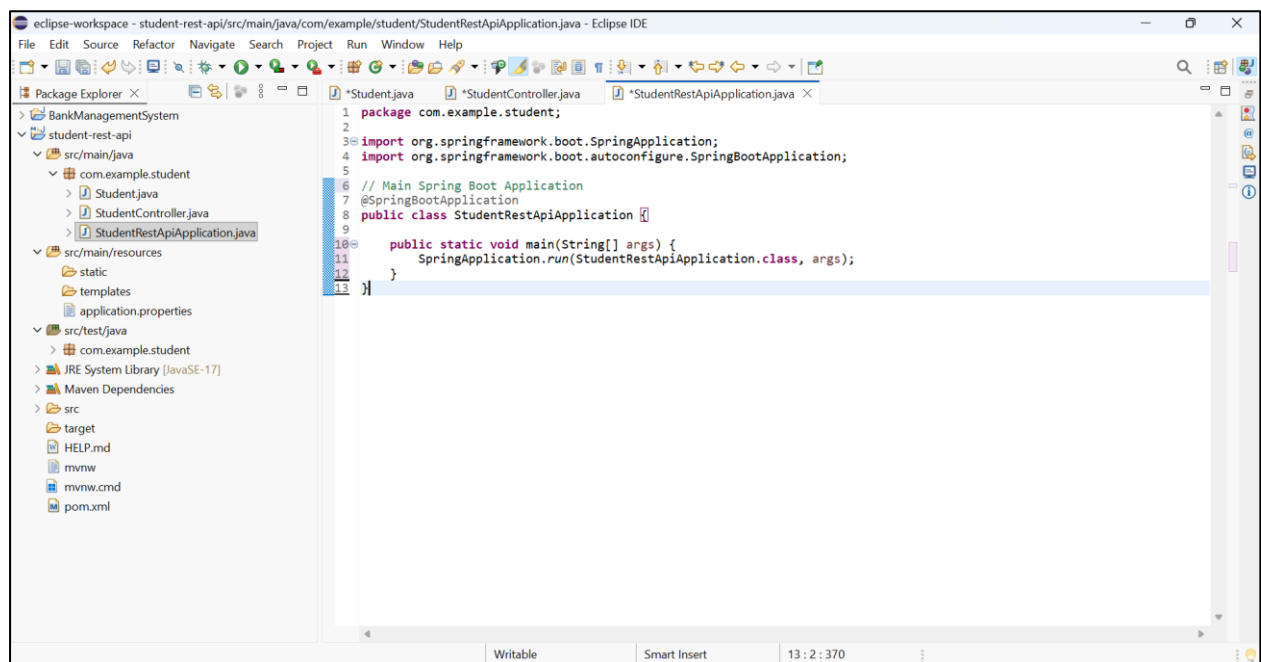
```

package com.example.student;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

// Main Spring Boot Application
@SpringBootApplication

```

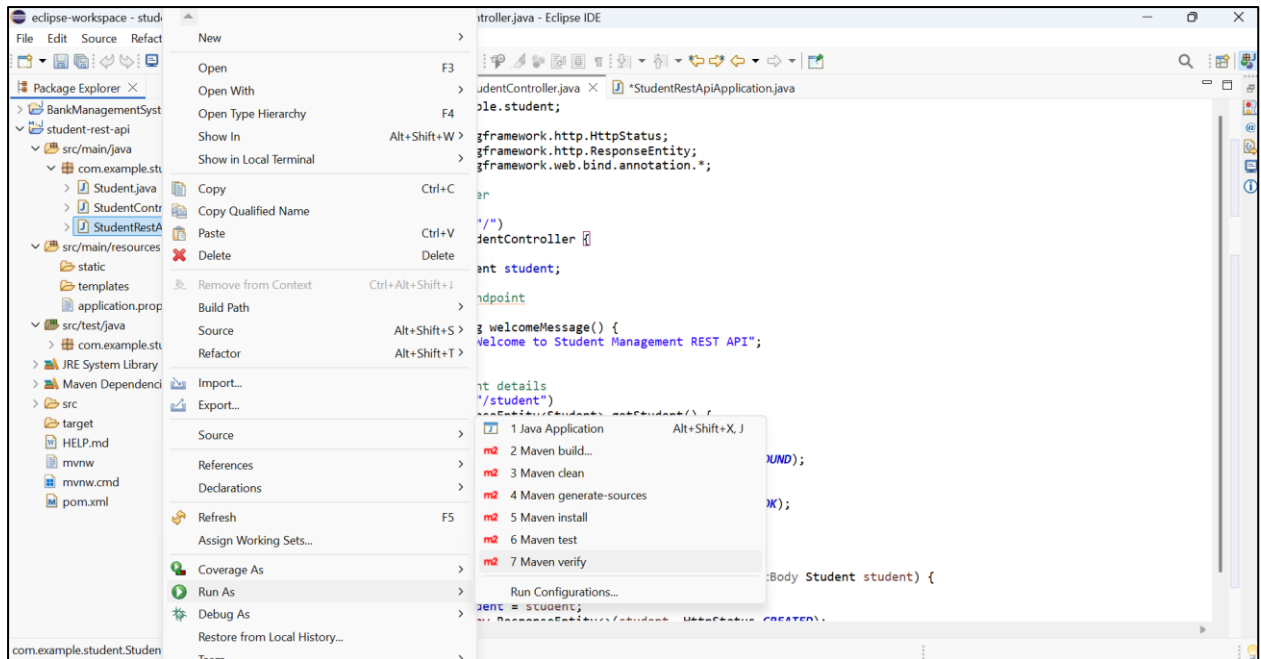
```
public class StudentRestApiApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(StudentRestApiApplication.class, args);  
    }  
}
```



Running the Application in Eclipse IDE

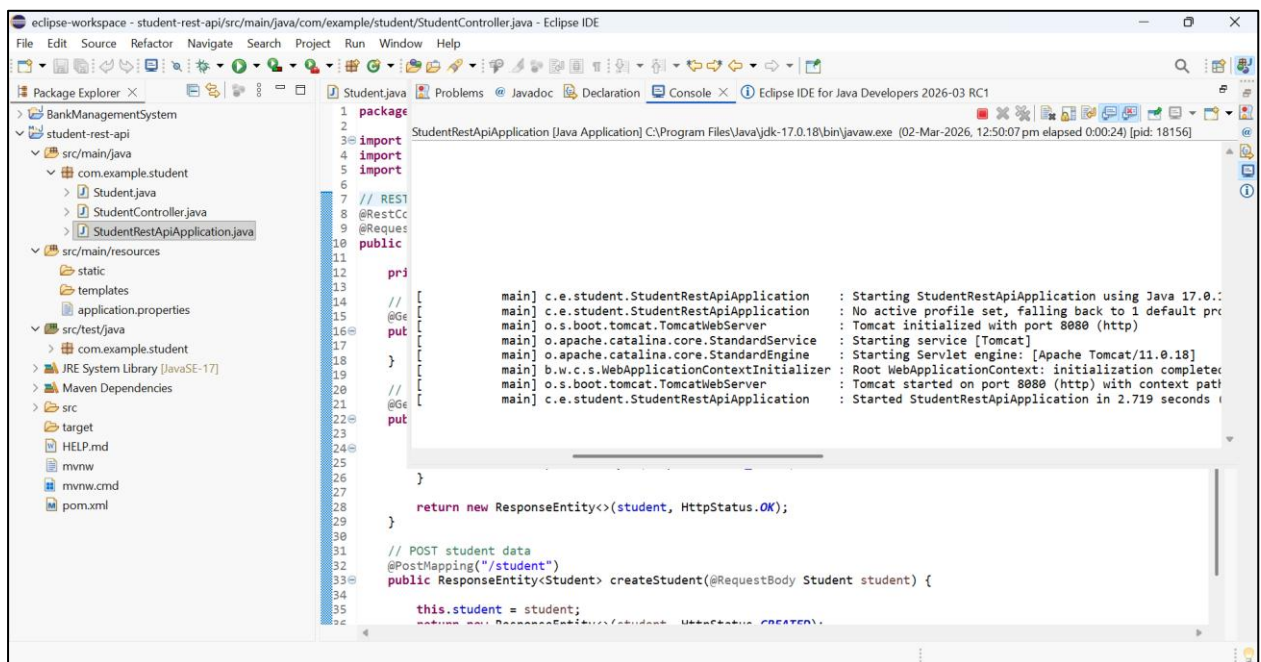
1. Right click the project
2. Click

Run As → Spring Boot App or Java Application



Spring Boot server will start on:

<http://localhost:8080>



Spring Boot console running

Testing the API

Welcome API

Browser:

<http://localhost:8080/>

Output:

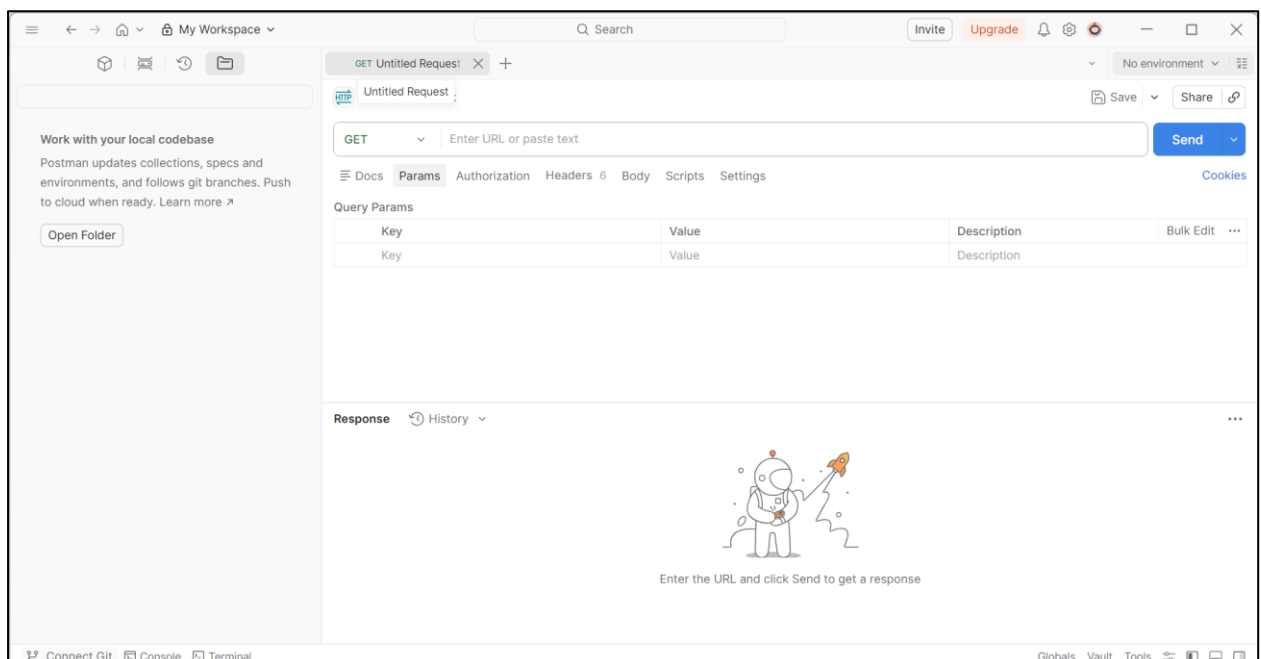


Welcome to Student Management REST API

GET API (If data is post already)

Browser:

<http://localhost:8080/student>



Output (JSON):

No output available yet.

Experiment 12

Title:

CRUD Operations using Spring Boot REST API

Problem Statement:

Implement **Create, Read, Update, and Delete (CRUD)** operations using **Spring Boot REST API**. Create an **Employee Management System REST API** that:

1. Adds a new employee
2. Retrieves all employees
3. Retrieves employee by ID
4. Updates employee details
5. Deletes employee record

Objectives:

After completing this experiment, students will be able to:

- Create **REST APIs using Spring Boot**
- Perform **CRUD operations**
- Connect Spring Boot with **database using JPA**
- Use **@RestController, @Service, and @Repository**
- Test APIs using **Postman**

Theory:

CRUD operations are the basic operations performed on data in applications.

Operation	HTTP Method	Description
Create	POST	Add new data
Read	GET	Retrieve data
Update	PUT	Modify existing data
Delete	DELETE	Remove data

Spring Boot Layers

1. **Controller Layer**
Handles HTTP requests.
2. **Service Layer**
Contains business logic.
3. **Repository Layer**
Communicates with the database using **Spring Data JPA**.

Using these layers ensures **clean architecture and maintainable code**.

Algorithm:

1. Start the Spring Boot project.
2. Add dependencies:

- Spring Web
- Spring Data JPA
- Database driver (H2/MySQL)
- 3. Create an **Employee entity class**.
- 4. Create **EmployeeRepository interface** extending JpaRepository.
- 5. Create **EmployeeService class** for business logic.
- 6. Create **EmployeeController class**.
- 7. Implement endpoints:
 - POST → Add employee
 - GET → Retrieve all employees
 - GET by ID → Retrieve specific employee
 - PUT → Update employee
 - DELETE → Remove employee
- 8. Connect the application to the database.
- 9. Run the application.
- 10. Test API endpoints using **Postman**.
- 11. Verify responses.
- 12. End the program.

Program Code:

Steps to Create Project using Spring Initializr

Go to:

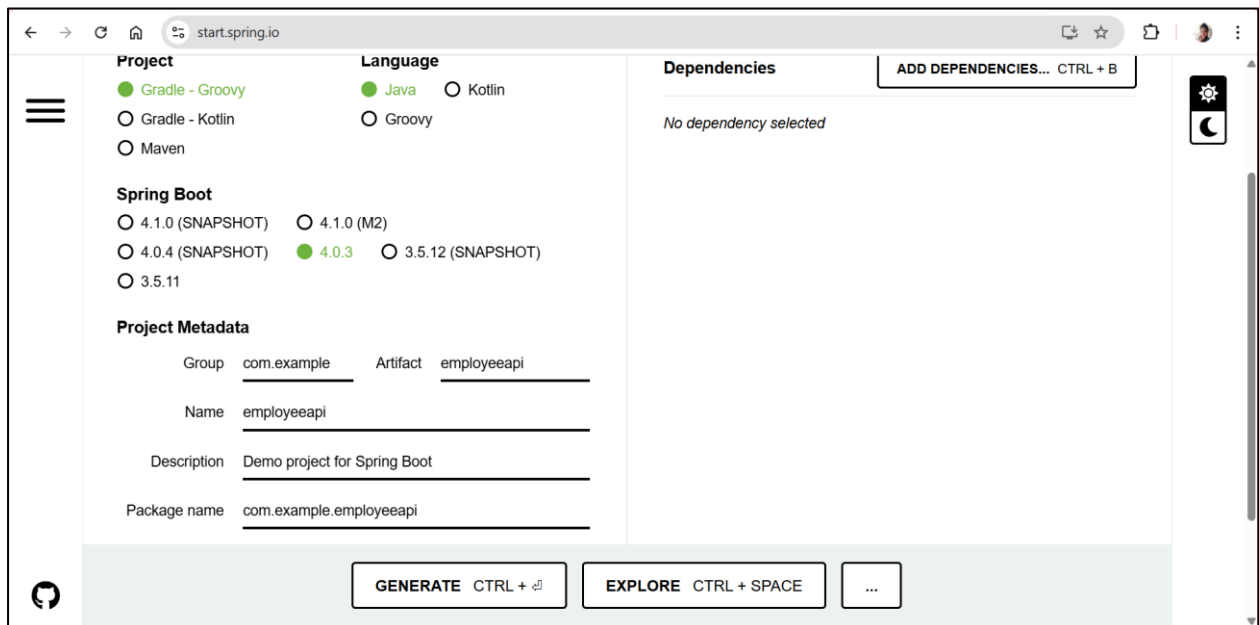
<https://start.spring.io>

The screenshot shows the Spring Initializr web application interface. The browser address bar displays 'start.spring.io'. The page features a sidebar with a hamburger menu and a settings icon. The main content area is divided into sections for Project, Language, Dependencies, Spring Boot, and Project Metadata. The Project section has radio buttons for Gradle - Groovy (selected), Gradle - Kotlin, and Maven. The Language section has radio buttons for Java (selected) and Kotlin. The Dependencies section has a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. The Spring Boot section has radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (M2) (selected), 4.0.4 (SNAPSHOT), 4.0.3 (selected), and 3.5.12 (SNAPSHOT). The Project Metadata section has input fields for Group (com.example) and Artifact (demo). At the bottom, there are buttons for 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a three-dot menu.

Project Configuration:

Field	Value
Project	Maven
Language	Java
Spring Boot	Latest

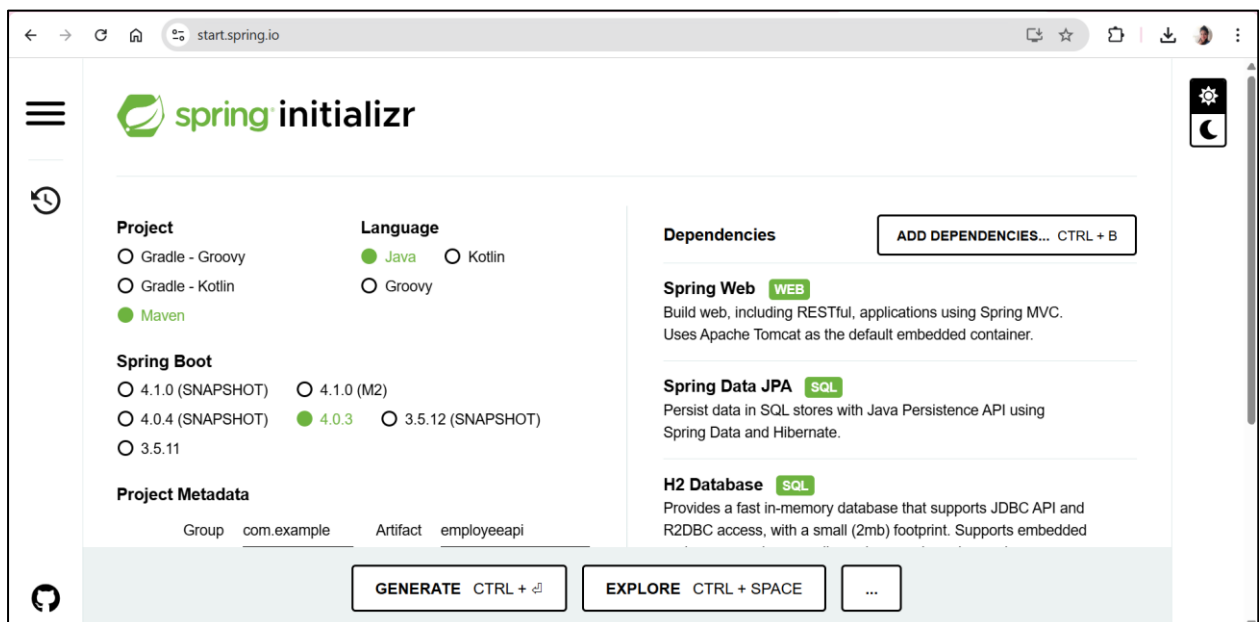
Group com.example
Artifact employeeapi
Java 17



The screenshot shows the start.spring.io web interface. On the left, there is a sidebar with a hamburger menu and a refresh icon. The main content area is divided into three sections: Project, Language, and Dependencies. The Project section has radio buttons for Gradle - Groovy (selected), Gradle - Kotlin, and Maven. The Language section has radio buttons for Java (selected), Kotlin, and Groovy. The Dependencies section has a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. Below these sections is the 'Spring Boot' section with radio buttons for versions: 4.1.0 (SNAPSHOT), 4.1.0 (M2), 4.0.4 (SNAPSHOT), 4.0.3 (selected), and 3.5.12 (SNAPSHOT). The 'Project Metadata' section has input fields for Group (com.example), Artifact (employeeapi), Name (employeeapi), Description (Demo project for Spring Boot), and Package name (com.example.employeeapi). At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and '...'.

Dependencies:

Spring Web
Spring Data JPA
H2 Database



The screenshot shows the start.spring.io web interface with dependencies added. The Project section has radio buttons for Gradle - Groovy, Gradle - Kotlin, and Maven (selected). The Language section has radio buttons for Java (selected), Kotlin, and Groovy. The Dependencies section has a button 'ADD DEPENDENCIES... CTRL + B' and three dependencies listed: 'Spring Web WEB' (Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.), 'Spring Data JPA SQL' (Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.), and 'H2 Database SQL' (Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded). The 'Project Metadata' section has input fields for Group (com.example), Artifact (employeeapi), Name (employeeapi), Description (Demo project for Spring Boot), and Package name (com.example.employeeapi). At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and '...'.

Click **Generate**.

Import Project in Eclipse IDE

1. Open Eclipse
2. Click

Testing API using Postman

Open postman

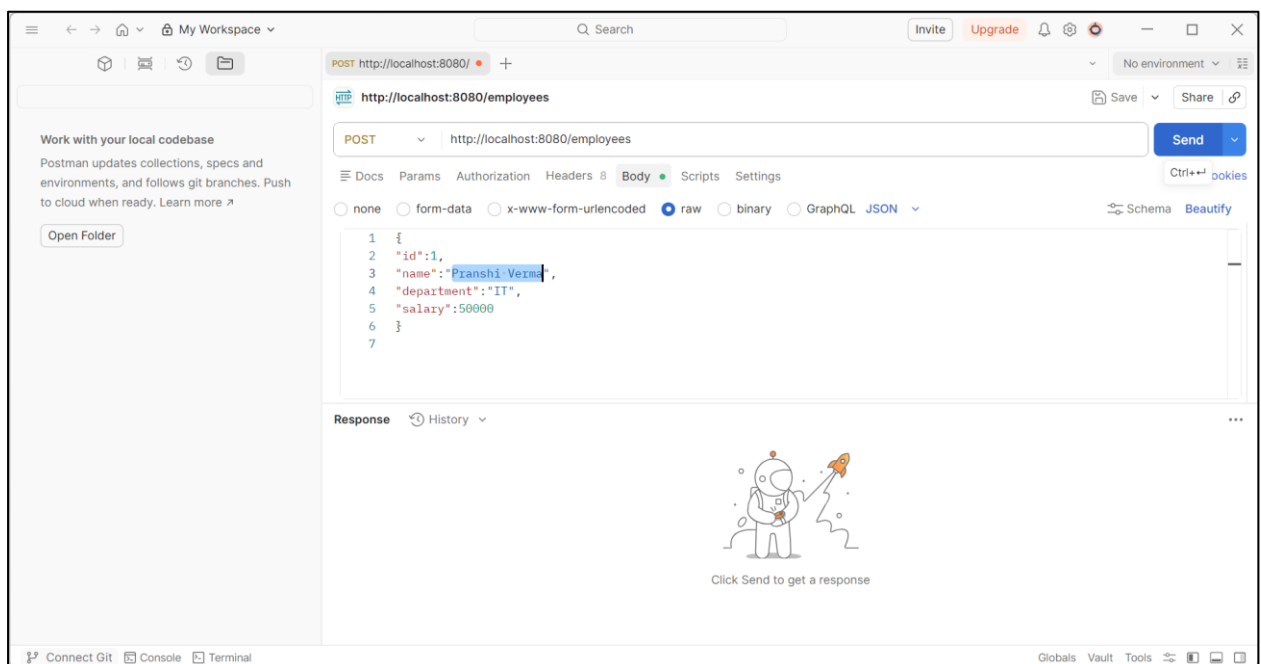
Create Employee (POST)

URL

`http://localhost:8080/employees`

Body

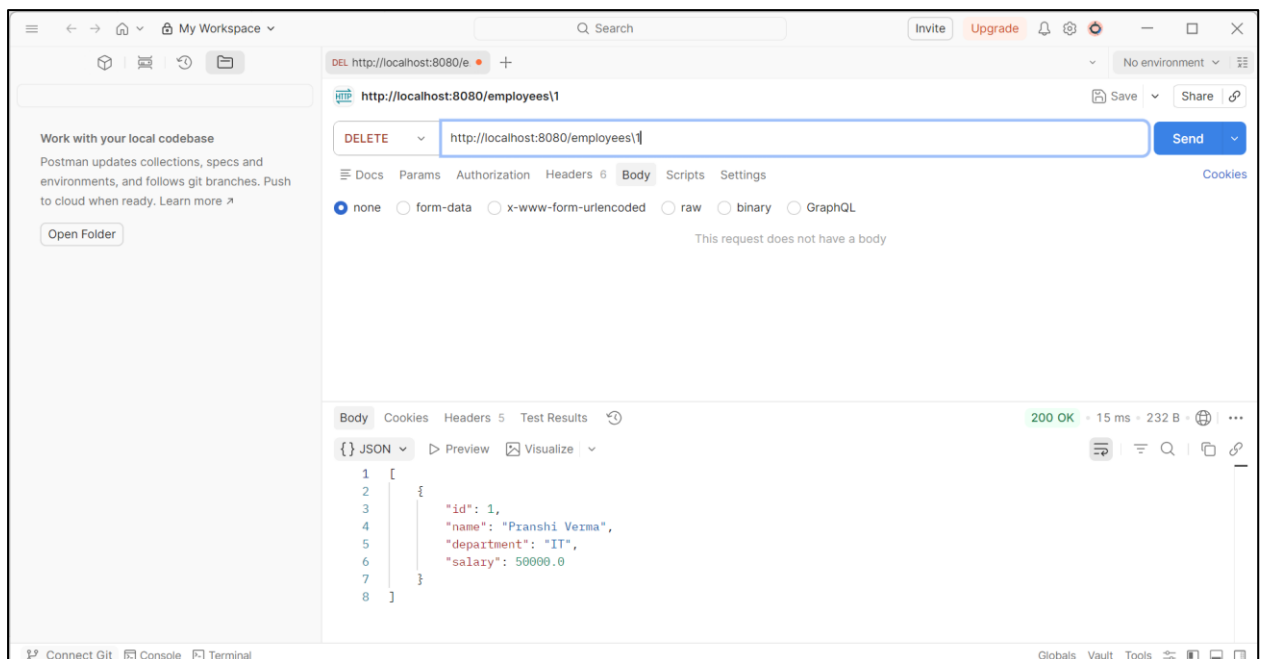
```
{  
  "id":1,  
  "name":"Pranshi Verma",  
  "department":"IT",  
  "salary":50000  
}
```

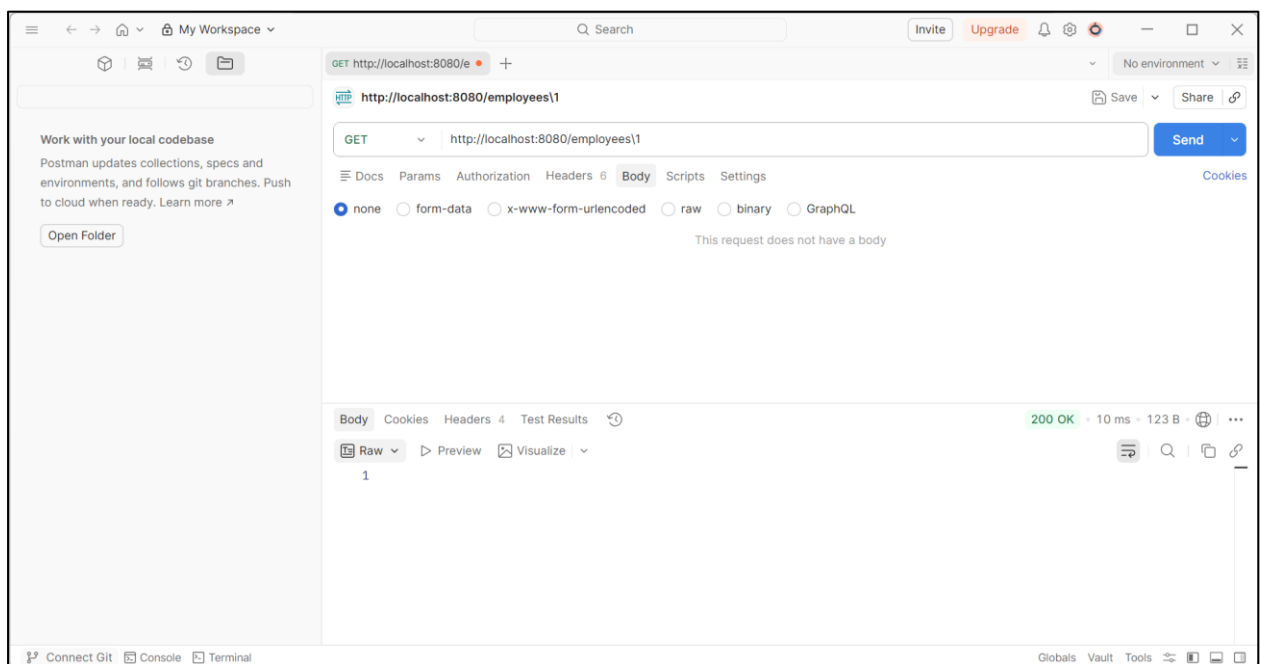
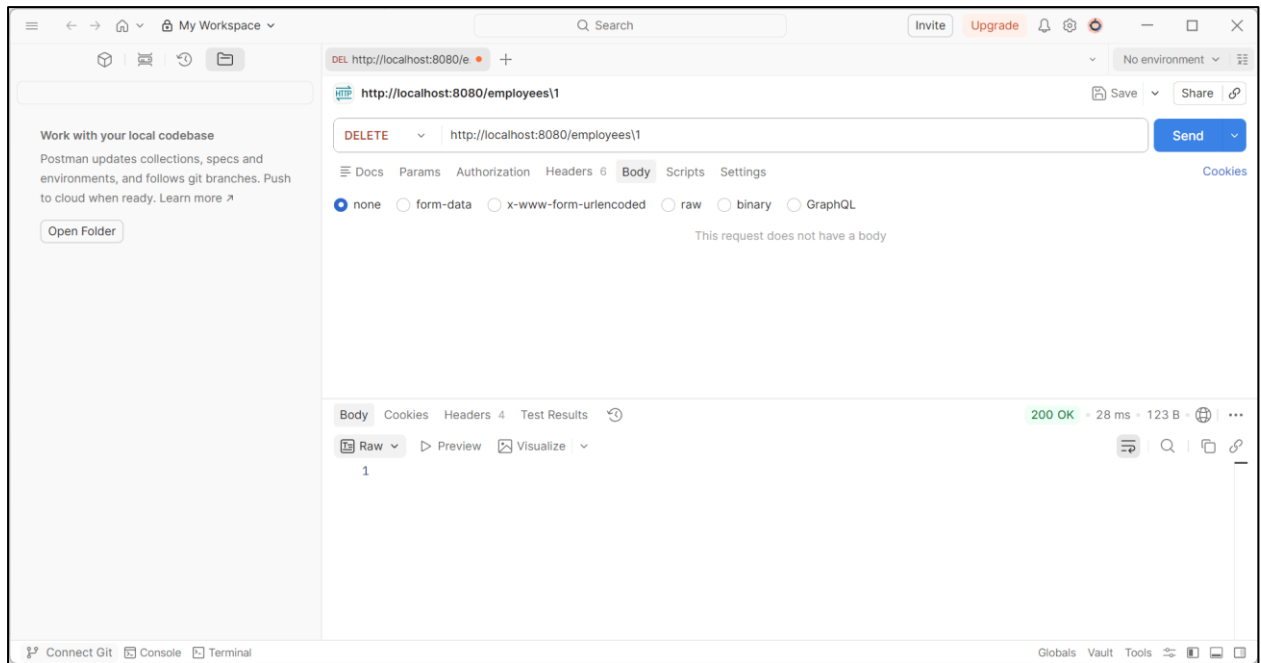




Delete Employee

<http://localhost:8080/employees/1>





No Data is available after successfully deletion.

Expected Outcome

Outcome:

Students successfully implemented **CRUD operations** using **Spring Boot REST APIs** and connected the application with a database using **Spring Data JPA**.